

An Empirical Study of Waterfall-style Multi-Agent Workflows for Class-Level Code Generation

Wasique Islam Shafin

Software Performance, Analysis, and Reliability (SPEAR)
Lab, Concordia University
Montreal, Canada
wi_shafin@encs.concordia.ca

Zhenhao Li

York University
Toronto, Canada
lzhenhao@yorku.ca

Md Nakhla Rafi

Software Performance, Analysis, and Reliability (SPEAR)
Lab, Concordia University
Montreal, Canada
r_mdnakh@encs.concordia.ca

Tse-Hsun (Peter) Chen

Software Performance, Analysis, and Reliability (SPEAR)
Lab, Concordia University
Montreal, Canada
peterc@encs.concordia.ca

Abstract

Modern software systems demand code that is not only functional but also maintainable and well-structured. While Large Language Models (LLMs) show potential in automating such development, most studies evaluate isolated, single-agent generation at the function level. This paper investigates how process structure and role specialization influence multi-agent LLM workflows for class-level code generation. We simulate a Waterfall-style software development life cycle spanning Requirement, Design, Implementation, and Testing using three LLM models (GPT-4o-Mini, DeepSeek-Chat, and Claude-3.5-Haiku) on 100 Python tasks from the ClassEval benchmark. Our results reveal that multi-agent workflows restructure, rather than uniformly improve, performance. Waterfall-style workflow yields cleaner and more maintainable code, yet often lowers functional correctness (−37.8% for GPT-4o-Mini, −39.8% for DeepSeek-Chat), except for Claude-3.5-Haiku (+9.5%). Crucially, process constraints shift the nature of failures, reducing structural issues like missing code but amplifying semantic and validation errors. Among activities, Testing has the greatest influence, strengthening verification but introducing reasoning faults, whereas Requirement and Design have a limited effect. Overall, this study provides the first empirical evidence that software process structure fundamentally shapes how LLMs reason and fail, highlighting trade-offs between disciplined workflow control and adaptive problem-solving in multi-agent code generation.

Keywords

large language model, code generation, agents

ACM Reference Format:

Wasique Islam Shafin, Md Nakhla Rafi, Zhenhao Li, and Tse-Hsun (Peter) Chen. 2026. An Empirical Study of Waterfall-style Multi-Agent Workflows for Class-Level Code Generation. In *22nd International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '26)*.



This work is licensed under a Creative Commons Attribution 4.0 International License. *PROMISE '26, Montreal, QC, Canada*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2584-5/26/07
<https://doi.org/10.1145/3803846.3807461>

July 5, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 10 pages.
<https://doi.org/10.1145/3803846.3807461>

1 Introduction

Large Language Models (LLMs) have become increasingly influential in Software Engineering (SE), assisting in diverse tasks such as code generation [9, 19, 27, 28, 31, 34], code completion and assistance [1, 12, 17, 37], fault localization and testing [22, 35, 38], and automated program repair [6, 7, 41, 43, 44, 46]. General-purpose models such as ChatGPT [33], Claude [4], and DeepSeek [13] have further extended the reach of LLMs into broader SE workflows, including debugging, documentation, and architecture planning [21, 47, 48]. As software systems continue to grow in complexity, the ability of LLMs to generate maintainable, reliable, and secure code has become central to their practical deployment in SE [2, 14].

Early advances in automated code generation primarily focused on function-level synthesis, where models such as Codex [11], AlphaCode [26], and CodeT5+ [39] demonstrated strong capabilities in producing short, self-contained functions. This line of research evolved through increasingly complex benchmarks like HumanEval [11], MBPP [5], and BigCodeBench [49], which standardize evaluations using metrics such as Pass@k. More recent frameworks, like Chain-of-Programming [20], interactive test-driven generation [16], and reasoning-enhanced prompting [25, 40], extend these capabilities toward more realistic and process-aware programming tasks. However, while these methods improve functional correctness, most evaluations still consider isolated functions rather than interconnected classes that reflect real-world software modularity and maintainability concerns.

To address this limitation, class-level benchmarks like *ClassEval* [15] have emerged to capture object-oriented code generation, assessing how well models handle multi-method dependencies and shared class state. Subsequent extensions such as Xue et al. [42] and Chen et al. [10] expand ClassEval in multiple languages and integrate error analysis frameworks, offering deeper insight into reasoning-based code generation. However, despite these advances, the majority of research remains focused on single-LLM paradigms. At the same time, multi-agent frameworks such as ChatDev [34], MetaGPT [19], SOEN-101 [27], and AgileCoder [32] have introduced collaborative, role-based approaches that emulate human

software processes by assigning distinct agents to roles such as Product Manager, Architect, Developer, and Tester. These systems have shown improvements in modularity, reasoning, and error mitigation at the function level. However, evaluations have not yet extended to class-level tasks, leaving an open question: *Do multi-agent LLM workflows enhance class-level code quality and correctness, and which development activities matter most?*

In this paper, we bridge these two research directions by empirically studying how a multi-agent workflow can operate within a classical Waterfall software process model for class-level code generation using *ClassEval*. We use Waterfall because its sequential structure (Requirement → Design → Implementation → Testing) makes it easier to study the effect of each development activity.

Our study assigns distinct roles (Requirement Engineer, Architect, Developer, and Tester) to specialized LLM agents, allowing us to investigate how each stage of the Waterfall process contributes to the final implementation. Through an ablation-based experimental design, we evaluate three LLM backends (GPT-4o-Mini, DeepSeek-Chat, Claude-3.5-Haiku) and compare them against direct prompting as the baseline, measuring (1) functional correctness (pass@1), (2) software quality and maintainability using SonarQube metrics, (3) identify frequently encountered error types, and (4) detailed failure categories through a taxonomy-driven error analysis.

Our results show that multi-agent LLM workflows do not uniformly enhance class-level code correctness but significantly affect code quality and error characteristics. Waterfall-style workflows give cleaner, more maintainable code; however, pass@1 accuracy declines for GPT-4o-Mini (-37.8%) and DeepSeek-Chat (-39.8%), while Claude-3.5-Haiku improves (+9.5%), indicating model-dependent effects. These workflows also increase runtime errors by 10-53%, with *ValueError*, *AssertionError*, and *TypeError* occurring most frequently. Although structural issues such as *Missing Code* are reduced, *Semantic Failure* and *Return Mismatches* become more common. Among development activities, Testing has the strongest influence: including it improves verification but increases reasoning and validation errors; excluding it yields the poorest correctness yet highest code quality. In contrast, the Requirements and Design stages have minimal impact on correctness or error types. The complete Waterfall configuration produces the lowest correctness and greatest adaptability concerns, suggesting that over-structuring the workflow constrains the flexibility of the final code. Incorporating these development activities into a multi-agent workflow enforces stricter validation and verification, but this also makes the code more error-prone and reduces its overall adaptability, intentionality, and correctness.

Overall, this paper makes the following contributions:

- We propose the first systematic empirical study of multi-agent LLM workflows applied to class-level code generation using *ClassEval* in the form of a Waterfall process model.
- We perform an empirical comparison of Waterfall-activity ablations that isolates the effect of every development activity on functional correctness and code quality.
- We found that multi-agent LLM workflows do not consistently improve class-level code correctness but do significantly influence code quality; testing has the greatest impact, strengthening verification while also increasing reasoning

errors, whereas overly structured Waterfall workflows reduce both adaptability and correctness.

- We identify the most frequent error patterns in multi-agent code generation and perform a detailed root-cause analysis, revealing which development activities are most responsible for introducing or mitigating these failures.
- We make all code and data from our experiments publicly available to enable reproduction, verification, and further research. [3]

2 Background and Related Work

LLMs have rapidly advanced the field of automated code generation, with growing research spanning model development, benchmarking, and multi-agent collaboration frameworks. This section first reviews the evolution of LLM-based code generation, then discusses recent progress in agent-driven workflows that simulate software engineering processes. We next summarize existing benchmarks and surveys, particularly those addressing class-level generation tasks. Finally, we identify the research gap motivating our study, which evaluates class-level code generation within structured multi-agent workflows, a dimension that remains missing in current literature.

2.1 LLM in Code Generation

The evolution of LLMs for code generation has developed rapidly since 2021, beginning with Codex [11], which demonstrated that models fine-tuned on large-scale code bases can translate natural language into executable code. Subsequent models, such as AlphaCode [26] and CodeT5+ [39], expanded further by handling competitive programming challenges and refining generation through encoder-decoder architectures and feedback-based ranking mechanisms. By 2023–2024, works like PANGU-CODER2 [36] and query-aligned prompting methods [25, 29, 40] improved model reasoning and task alignment. More recent research has shifted toward embedding LLMs within full software workflows, incorporating test-driven development and interactive debugging [16, 20]. Surveys published in 2025 [2, 14] highlight a transition from code snippet generation to end-to-end software development workflows, encompassing planning, optimization, and integration.

2.2 Multi-Agent Code Generation Frameworks

In parallel, multi-agent frameworks have emerged to emulate collaborative software engineering processes using role-specialized LLMs. Frameworks like ChatDev [34] and MetaGPT [19] assign agents roles such as Product Manager, Developer, and Tester, coordinating through dialogue (chatting) to design, implement, and test software solutions. These systems primarily operate at the function level, evaluating outcomes via completeness, executability, and pass@k metrics. SOEN-101 [27] extends this approach by structuring multi-agent workflows according to software process models (e.g., Waterfall, TDD, Scrum), while MAS [8] categorizes common failure types across frameworks, such as system design errors and inter-agent misalignments. AgileCoder [32] further introduces Agile-inspired iterative sprints, highlighting adaptability and incremental refinement. Collectively, these studies confirm that agent-based LLMs improve task division and communication in

software generation, but they remain confined to function-level code, leaving class-level software development largely unexplored.

2.3 Benchmarks and Surveys

Benchmarking remains central to evaluating progress in LLM-based code generation. Widely adopted datasets such as HumanEval [11], MBPP [5], and BigCodeBench [49] measure correctness using the pass@k metric, focusing on isolated, function-level tasks. Surveys by Jiang et al. [21], Zhang et al. [47], and Zheng et al. [48] systematically compare LLM performance across these benchmarks, revealing ongoing challenges in reasoning, building complex code, and error propagation. Recognizing the limitations of function-level assessments, ClassEval [15] was introduced to evaluate class-level generation tasks involving interdependent methods, shared variables, and internal class logic. Follow-up works such as Xue et al. [42] expanded ClassEval to Java and C++, incorporating new metrics and error analysis, while Chen et al. [10] examined reasoning performance across HumanEval and ClassEval. Complementary frameworks like CoCoST [18] integrate automated test generation and web search to assess broader problem-solving capabilities. These advances collectively provide a foundation for studying class-level code generation, yet none explore how structured, agent-driven workflows like waterfall influence performance on such complex tasks.

2.4 Limitations and Research Gap

While multi-agent frameworks have proven effective for function-level collaboration and class-level benchmarks have captured higher program complexity, these two directions remain largely disconnected. Existing literature does not examine how structured agent workflows affect class-level code generation quality, nor how different software development activities (requirements analysis, design, implementation, and testing) contribute to correctness and maintainability.

Our study bridges this gap by evaluating class-level code generation through a Waterfall-inspired multi-agent workflow. This structured setup enables activity-wise analysis of LLM behaviour by tracing how each development activity impacts the final code quality across models. In contrast to prior work emphasizing model architecture or function-level benchmarks, our contribution lies in characterizing how collaborative process design has an impact on what LLMs produce, offering new insights into class-level code generation that is aware of the software engineering process.

3 Methodology and Experiment Setup

To understand how different development activities influence the quality of code produced during *class-level* code generation, we use agents to emulate the **Waterfall** model. Its straightforward and sequential nature allows us to examine how individual development activities affect the resulting code. We follow a step-by-step linear process, where development flows downwards through distinct phases (from requirements to design, implementation, and testing).

Figure 1 provides an overview of our study design. Our experiment design is organized around two main components: (1) **Agent Roles**, which define the LLM agents and their responsibilities, and (2) **Agent Communication**, which describes how these agents

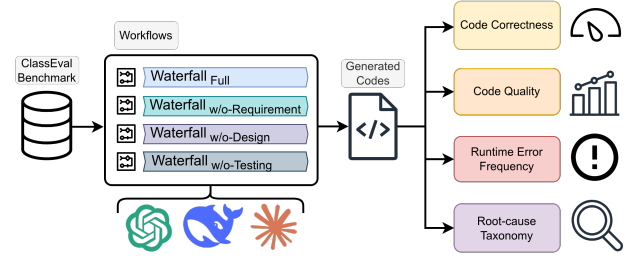


Figure 1: Approach Overview

interact throughout the development phases under the Waterfall model.

3.1 Agent Roles

We employ four LLM agents to represent the primary activities of the software development life cycle (SDLC) within a *Waterfall* process: *requirements*, *design*, *implementation*, and *testing*. These agents are (1) **Requirement Engineer**, (2) **Architect**, (3) **Developer**, and (4) **Tester**, each focusing on a specific development activity and their interactions, mirror their real-life counterparts.

Every agent uses the same unified prompt template, with differences in role-specific details. We adapt and modify the prompt structure from SOEN-101 [27] to enable class-level generation workflows, whereas it focused only on function-level generation only. The prompt template defines how each agent interprets its assigned role, consumes relevant context, and generates the corresponding software artifact. A simplified example of the template is shown below:

```

1 {
2   "Role": "You are a [role] delegated for [task]",
3   "Instruction": "According to the Context,
4     ↳ [role-specific-instruction]",
5   "Example": "[document example]",
6   "Context": "[context]",
7   "Question": "Follow the Instruction. The [document] must
8     ↳ satisfy the requirements."
9 }
```

role defines the agent's role in the process model (e.g., Developer or Tester), *task* specifies the objective of that role, *document* represents the artifact to be produced, *example* provides a reference format, *role-specific-instruction* offers detailed guidance tailored to the task, and *context* supplies necessary inputs or artifacts from earlier stages.

Table 1 summarizes the *tasks*, *instructions*, and *contexts* assigned to each agent, showing how they collaborate to generate, refine, and verify software artifacts. By delegating well-defined tasks to specialized agents, our approach enables systematic experimentation within a Waterfall workflow, refining how agents collaborate and exchange information across sequential phases.

3.2 Agent Communication

For our study, the agent communication is based on the Waterfall process model. This process model follows a fixed, linear sequence of well-defined activities (e.g., requirements, design, implementation, and testing), where each activity depends strictly on the

Table 1: Tasks, instructions, and context associated with each role prompt

Role	Task	Instructions	Context
Requirement Engineer	Analyze the task description and write a requirements document.	1) Review the task description. 2) Write a requirements document that accounts for method signatures and class variables.	Task description (user requirement).
Architect	Design the high-level components and structure of the code.	1) Review the provided documents. 2) Write a design document that preserves method definitions and serves as a guide for developers.	Requirements document.
Developer	Implement Python code that meets all requirements.	1) Review the provided documents. 2) Write clean, efficient, and readable code following best practices. Use a single class structure and keep method definitions unchanged.	Design document.
	Fix code while ensuring all requirements are met.	1) Review the test reports and provided documents. 2) Revise the code to make it efficient, readable, and consistent with best practices.	Test report and original code.
Tester	Design test cases that verify all requirements.	1) Review the provided documents. 2) Write test cases that preserve method definitions, cover normal, edge, and error conditions, and include at least five test cases per method.	Task description (user requirement).
	Write a Python test script using the unittest framework.	1) Review the provided documents. 2) Write test scripts with one test case for each scenario, reusing existing modules where possible.	Test case document.
	Write a test failure report.	1) Review the test execution results. 2) Document the outcomes in a test report.	Test execution results.

output of the previous one. Agents are assigned specialized roles and operate in isolated activities with formal handoffs, rather than overlapping work. This strict separation of activities makes it easier to analyze whether the activity itself is effective. Within each activity, agents also refine their outputs using feedback [30].

This section describes how agents coordinate within these structured activities and how their interactions are organized into a complete Waterfall-inspired workflow.

3.2.1 Communication. The agent communication process is divided into four sequential activities: *Requirement*, *Design*, *Implementation*, and *Testing*, corresponding directly to the core stages of the *Waterfall* software development life cycle (SDLC). As illustrated in Figure 2, these four activities collectively form a complete Waterfall-inspired communication process among the LLM agents. **Requirement Activity.** The *Requirement Engineer* agent begins the process by transforming the User Requirement into a formal Requirement Document that captures the high-level functional and structural details, such as method signatures, class variables, and inter-dependencies. Communication in this phase primarily involves clarification and validation of user needs by the agent. The document is self-refined [30] through feedback from the *Architect* and *Tester* to ensure fulfillment of acceptance criteria before progressing to the next phase.

Design Activity. The *Architect* agent receives the completed Requirement Document and produces a Design Document that outlines the system’s architecture, class structure, and method interactions. This phase formalizes the blueprint for development, and feedback from the *Developer* and *Tester* is incorporated [30] only to resolve inconsistencies or design ambiguities. This phase reflects Waterfall’s controlled, document-driven communication before implementation begins.

Implementation Activity. The *Developer* agent implements the Design Document by producing a Code Document that the agent realizes the required functionality in executable code using a single class structure. Like that of previous phases, our framework allows self-refinement [30] based on suggestions from the *Architect* and *Tester* to correct implementation deviations or improve maintainability.

This controlled interaction simulates real-world review cycles that occur before code is formally handed over for testing.

Testing Activity. The *Tester* agent derives Test Cases directly from the User Requirement and translates them into executable Test Scripts. These test cases are self-refined [30] using feedback from the *Architect* and *Developer* to resolve ambiguity and ensure accurate test coverage. The *Tester* executes the Test Script against the Code Document, producing a Test Report that summarizes the detected faults and performance outcomes. Finally, the *Developer* may perform up to three bug-fix cycles based on this report, after which the process concludes. Thus, reflecting the validation and closure phase of the Waterfall cycle.

3.2.2 Studying the Effect of Each Development Activity. To understand the impact of each activity, we take the complete Waterfall and remove one activity at a time, and then observe how its absence affects overall performance, code quality, and the possible issues. This ablation-based approach allows us to identify which phases most strongly affect the correctness, completeness, and reliability of the final output. We retain the Implementation phase across all the configurations, as code generation is crucial to producing the executable artifact required for evaluation.

For our study, we have 4 configurations of Waterfall, where *w/o* denotes the removal of the corresponding activity: 1) **Waterfall_{Full}**, 2) **Waterfall_{w/oRequirement}**, 3) **Waterfall_{w/oDesign}**, 4) **Waterfall_{w/oTesting}**.

3.3 Dataset

We use the **ClassEval** dataset [15], a human-curated benchmark for evaluating LLMs on *class-level Python coding tasks*. Unlike prior benchmarks such as **HumanEval** [11] and **MBPP** [5], which assess short, function-level code generation, **ClassEval** emphasizes the generation of complete classes that integrate multiple methods, fields, and dependencies. This class structure introduces more reasoning challenges related to design consistency, state management, and inter-method interaction, i.e., features that closely resemble real-world software development than that of isolated coding exercises. The dataset contains 100 programming assignments, each

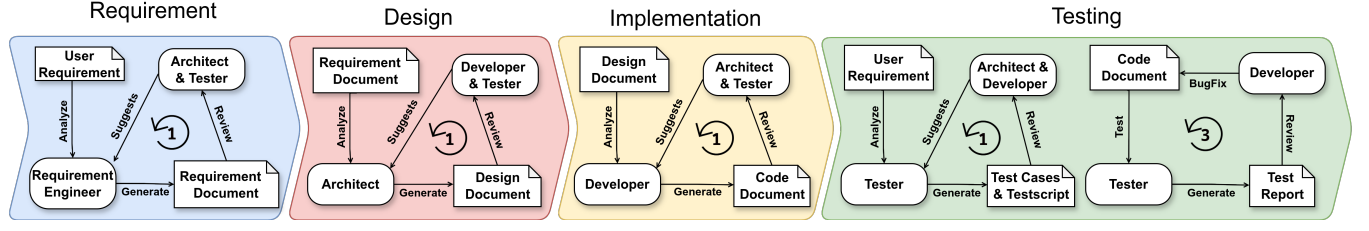


Figure 2: Activities of Waterfall Workflow

requiring the implementation of a complete Python class. It was developed over approximately 500 person-hours, emphasizing diversity and realism in software development scenarios.

Each ClassEval task provides: (1) *Class Skeleton*. This includes import statements, the class name, a brief description, and the class constructor, along with detailed specifications for each method. Each method specification outlines the method signature, functional description, parameters, and return values, and example input/output. These skeletons serve as the task descriptions in our study. (2) *Ground-Truth Implementation*. A complete and correct reference implementation of the target class in Python, serving as the canonical solution. (3) *Test Cases*. An average of 33.1 test cases per class to rigorously assess code correctness. These tests allow us to measure both function-level and class-level accuracy. We want to study how individual development activities affect different quality aspects at the class level. We therefore adopt a holistic class-generation strategy, producing the entire class at once, which has been shown to achieve better performance with modern LLMs [15, 42].

3.4 LLM Choice and Implementation Details

We evaluate our workflows using three contemporary LLMs: GPT-4o-Mini (gpt-4o-mini-2024-07-18), DeepSeek-Chat (DeepSeek-V3-0324), and Claude-3.5-Haiku (claude-3-5-haiku-20241022). All three models represent lightweight yet high-performance variants of their respective families, making them suitable for our multi-agent workflow-based code generation study. By applying identical workflows across models, we aim to observe whether different LLMs exhibit distinct sensitivities to various development activities in the Waterfall process model and to determine which phases of the Waterfall pipeline have the most significant impact on their performance. For all models, the temperature is set to 0.8 to strike a balance between creativity and determinism during generation.

4 Results

In this section, we present the results to our research questions.

4.1 RQ1: What is the accuracy of the generated code?

Motivation: Class-level code generation differs from method-level generation, as it requires handling higher-level design decisions, inter-method interactions, and overall class organization rather than isolated function logic. In this RQ, we study how applying the *Waterfall* process model and enforcing/implementing each of its development activities affect the quality of generated code, and what

trade-offs arise in terms of code correctness or quality compared to unstructured direct prompting.

Approach: For the class-level analysis of LLM agent-generated code, we compare Waterfall-inspired workflows against a *Raw-Prompt* baseline, where code is generated directly from task descriptions without intermediate stages. To isolate the impact of individual development activities, we evaluate variants in which each stage is removed in turns. We assess generated code along two dimensions: (1) **code correctness**, measured using *Pass@1* from the ClassEval benchmark, and (2) **code quality**, evaluated via static analysis with SonarQube. This combined approach captures both execution success and overall code quality.

Code Correctness. We use *Pass@k* [11, 23] as the primary correctness metric, representing the expected proportion of problems correctly solved given k generated code samples:

$$\text{Pass}@k = \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where n is the number of generated samples, c the number of correct solutions, and k the evaluated samples. We report **class-level** and **function-level** *Pass@1*. Class-level correctness requires all methods in a class to pass, while function-level correctness evaluates methods independently. Using *Pass@1* provides a strict single-attempt measure. All evaluations use the ClassEval framework.

Code Quality. To complement test-based correctness, we analyze code quality using **SonarQube**¹, which detects and categorizes issues into two major groups: *Software Quality* and *Clean Code Attributes*. The *Software Quality*² group includes (1) **Security** - Safeguarding software against unauthorized access, use, or damage (protection from misuse), (2) **Reliability** - Ensuring software consistently performs as expected over time (consistent performance), and (3) **Maintainability** - Ease of understanding, fixing, and improving software code (ease of modification). The *Clean Code*³ group evaluates (1) **Consistency** - Writing code in a uniform and conventional manner (adherence to conventions), (2) **Intentionality** - Ensuring code is precise and purposeful (clarity and completeness), (3) **Adaptability** - Structuring code for easy evolution and confident development (modular and reusable structure), and (4) **Responsibility** - Considering ethical obligations and societal impact in code (lawful and ethical coding behavior).

¹<https://www.sonarsource.com/products/sonarqube/>

²<https://docs.sonarsource.com/sonarqube-server/10.6/user-guide/clean-code/software-quality>

³<https://docs.sonarsource.com/sonarqube-server/10.6/user-guide/clean-code/definition>

Table 2: Comparison of GPT-4o-Mini, DeepSeek-Chat, and Claude-3.5-Haiku under Waterfall variants. Each cell reports the raw metric value and its percentage change relative to the RawPrompt baseline. Green denotes improvement and red denotes regression. Higher values indicate better performance for Pass@1, while lower values indicate better performance for Software Quality and Clean Code Attributes.

Model	Approach	Pass@1 (↑)		Software Quality (↓)		Clean Code Attribute (↓)		
		Class	Function	Reliability Issues	Maintainability Issues	Consistency Issues	Intentionality Issues	Adaptability Issues
GPT-4o-Mini	RawPrompt	0.35	0.6853	0	0.2254	0.1211	0.0908	0.0135
	Waterfall _{Full}	0.21 (-40%)	0.5478 (-20%)	0.0044	0.2188 (-3%)	0.0774 (-36%)	0.0796 (-12%)	0.0663 (+391%)
	Waterfall _{w/o-Requirement}	0.23 (-34%)	0.5458 (-20%)	0.0023	0.1991 (-12%)	0.0801 (-34%)	0.0618 (-32%)	0.0595 (+341%)
	Waterfall _{w/o-Design}	0.21 (-40%)	0.5896 (-14%)	0.0072	0.2141 (-5%)	0.0818 (-33%)	0.0987 (+9%)	0.0409 (+203%)
	Waterfall _{w/o-Testing}	0.22 (-37%)	0.5458 (-20%)	0	0.1820 (-19%)	0.0688 (-43%)	0.0738 (-19%)	0.0393 (+191%)
DeepSeek-Chat	RawPrompt	0.46	0.7529	0	0.2015	0.1085	0.0558	0.0372
	Waterfall _{Full}	0.32 (-30%)	0.6255 (-17%)	0.0059	0.2075 (+3%)	0.0889 (-18%)	0.0692 (+24%)	0.0514 (+38%)
	Waterfall _{w/o-Requirement}	0.29 (-37%)	0.6554 (-13%)	0.0109	0.2049 (+2%)	0.0916 (-16%)	0.0632 (+13%)	0.0567 (+52%)
	Waterfall _{w/o-Design}	0.31 (-33%)	0.6793 (-10%)	0	0.2001 (-1%)	0.0703 (-35%)	0.0575 (+3%)	0.0724 (+95%)
	Waterfall _{w/o-Testing}	0.19 (-59%)	0.5378 (-29%)	0.0040	0.1564 (-22%)	0.0842 (-22%)	0.0321 (-43%)	0.0421 (+13%)
Claude-3.5-Haiku	RawPrompt	0.21	0.3685	0	0.1659	0.0869	0.0435	0.0356
	Waterfall _{Full}	0.22 (+5%)	0.6255 (+70%)	0	0.1736 (+5%)	0.0825 (-5%)	0.0282 (-35%)	0.0629 (+77%)
	Waterfall _{w/o-Requirement}	0.24 (+14%)	0.6056 (+64%)	0.0024	0.1591 (-4%)	0.0807 (-7%)	0.0546 (+26%)	0.0261 (-27%)
	Waterfall _{w/o-Design}	0.29 (+38%)	0.6275 (+70%)	0	0.1734 (+5%)	0.0759 (-13%)	0.0477 (+10%)	0.0499 (+40%)
	Waterfall _{w/o-Testing}	0.17 (-19%)	0.4343 (+18%)	0.0021	0.1501 (-10%)	0.0813 (-6%)	0.0396 (-9%)	0.0292 (-18%)

Note: The Security and Responsibility columns are omitted from the table as all their values were zero. Reliability Issues, Maintainability Issues, Consistency Issues, Intentionality Issues, and Adaptability Issues are reported as issue counts; thus, negative changes indicate improvements and positive changes indicate regressions.

SonarQube reports issue density normalized per 10 non-comment lines of code (ncLOC), defined as:

$$\text{Issue Density per 10 ncLOC} = \frac{\text{SonarQube Issues}}{\text{Total ncLOC}} \times 10$$

Each workflow is analyzed as a separate SonarQube project. Higher issue density indicates poorer code quality, complementing test-based correctness results.

Results: Table 2 presents the comparative performance of all models under different Waterfall configurations.

Accuracy. *Waterfall variants reduce Pass@1 for GPT-4o-Mini and DeepSeek-Chat, but improve performance for Claude-3.5-Haiku.* Applying Waterfall workflows consistently lowers accuracy for GPT-4o-Mini and DeepSeek-Chat, where RawPrompt achieves the best results. Across variants, class-level accuracy drops by 30–40% and function-level accuracy by 10–25%, with w/o Testing performing worst. In contrast, Claude-3.5-Haiku benefits from structured prompting: most Waterfall variants outperform RawPrompt, with w/o Design achieving the highest accuracy and the full workflow yielding large function-level gains (+70%). Across all models, Pass@1 is substantially lower at the class level than at the function level, indicating the difficulty of generating fully correct classes.

Software Quality. *Reliability declines across all models under Waterfall workflows, while maintainability improves mainly for GPT-4o-Mini.* For GPT-4o-Mini, all Waterfall variants improve maintainability, most notably w/o Testing, but consistently reduce reliability. DeepSeek-Chat and Claude-3.5-Haiku show mixed maintainability outcomes depending on the variant, yet both also exhibit reduced reliability across all settings. This suggests that staged workflows improve organization but harm overall stability. No security issues were observed.

Clean Code Attributes. *Consistency improves under Waterfall*

workflows, while adaptability generally degrades, especially for smaller models. All models show fewer consistency issues under Waterfall prompting. However, GPT-4o-Mini and DeepSeek-Chat experience notable adaptability regressions, whereas Claude-3.5-Haiku shows more balanced outcomes, particularly under w/o Testing. Intentionality results are mixed across models. These trends suggest that smaller models may overflow workflow stages, improving consistency at the cost of adaptability. No responsibility issues were detected.

RQ1 Summary: Waterfall workflows reduce Maintainability and Consistency issues but increase Reliability, Adaptability, and Intentionality issues. They also lower Pass@1 scores at both class and function levels. Performance varies by model: GPT-4o-Mini and DeepSeek-Chat degrade, while Claude-3.5-Haiku improves significantly.

4.2 RQ2: What kinds of errors occur during class-level code generation?

Motivation: This research question investigates the types of errors that arise in class-level code generation under the Waterfall process model. While RQ1 showed that Waterfall workflows improve code quality and cleanliness, they also introduce runtime reliability issues and stage inconsistencies. Building on this, we analyze the runtime error categories observed across different LLMs and workflow variants to identify the sources of coordination failures.

Approach: We manually evaluated all 1,500 generated code samples (100 ClassEval tasks × 3 models × 5 workflows) by executing them against the corresponding ClassEval unit tests and recording

Table 3: Comparison of error types across three different models. Each value reports the raw count and relative change (%) compared to the model’s *RawPrompt* baseline across Waterfall variants.

Error Type	GPT-4o-Mini					DeepSeek-Chat					Claude-3.5-Haiku				
	Raw	Waterfall				Raw	Waterfall				Raw	Waterfall			
		Full	w/o-Req.	w/o-Des.	w/o-Test.		Full	w/o-Req.	w/o-Des.	w/o-Test.		Full	w/o-Req.	w/o-Des.	w/o-Test.
AssertionError	53	60 (+13%)	54 (+2%)	62 (+17%)	51 (-4%)	51	50 (-2%)	55 (+8%)	58 (+14%)	60 (+18%)	27	58 (+115%)	61 (+126%)	54 (+100%)	54 (+100%)
ValueError	7	25 (+257%)	24 (+243%)	22 (+214%)	28 (+300%)	7	24 (+243%)	17 (+143%)	15 (+114%)	29 (+314%)	3	22 (+633%)	17 (+467%)	17 (+467%)	25 (+733%)
TypeError	4	7 (+75%)	8 (+100%)	9 (+125%)	9 (+125%)	5	4 (-20%)	4 (-20%)	5	7 (+40%)	1	3 (+200%)	4 (+300%)	5 (+400%)	3 (+200%)
AttributeError	5	5	6 (+20%)	6 (+20%)	10 (+100%)	3	3	2 (-33%)	4 (+33%)	3	1	2 (+100%)	2 (+100%)	4 (+300%)	2 (+100%)
KeyError	0	4 (+400%)	4 (+400%)	2 (+200%)	3 (+300%)	1	3 (+200%)	2 (+100%)	3 (+100%)	3 (+100%)	0	2 (+200%)	4 (+400%)	5 (+500%)	1 (+100%)
FileNotFoundError	2	5 (+150%)	3 (+50%)	3 (+50%)	4 (+100%)	1	2 (+100%)	3 (+200%)	2 (+100%)	3 (+200%)	1	2 (+100%)	1	2 (+100%)	1
IndexError	2	4 (+100%)	0 (-100%)	0 (-100%)	1 (-50%)	2	0 (-100%)	0 (-100%)	0 (-100%)	1 (-50%)	1	1	0 (-100%)	1	0 (-100%)
NameError	3	2 (-33%)	1 (-67%)	1 (-67%)	3	2	1 (-50%)	2	2	2	51	3 (-94%)	2 (-96%)	2 (-96%)	20 (-61%)
SyntaxError	0	0	1 (+100%)	2 (+200%)	2 (+200%)	1	5 (+400%)	6 (+500%)	5 (+400%)	6 (+500%)	0	0	4 (+400%)	2 (+200%)	1 (+100%)
ZeroDivisionError	2	0 (-100%)	0 (-100%)	0 (-100%)	0 (-100%)	3	0 (-100%)	0 (-100%)	0 (-100%)	0 (-100%)	2	2	1 (-50%)	1 (-50%)	0 (-100%)
OperationalError*	0	1 (+100%)	2 (+200%)	2 (+200%)	1 (+100%)	1	2 (+100%)	0 (-100%)	0 (-100%)	1 (+100%)	0	1 (+100%)	2 (+200%)	1 (+100%)	1 (+100%)
Time Limit Exceeded	2	1 (-50%)	1 (-50%)	0 (-100%)	0 (-100%)	2	0 (-100%)	1 (-50%)	1 (-50%)	2	1	0 (-100%)	1	1	1
Exception	0	1 (+100%)	1 (+100%)	0	1 (+100%)	0	0	0	0	2 (+200%)	0	1 (+100%)	0	0	0
DeprecationError*	1	0 (-100%)	0 (-100%)	0 (-100%)	1	1	1	1	0 (-100%)	1	0	0	1	1	0
OSError	0	1 (+100%)	0	0	0	1	0 (-100%)	0	0	1	1	0 (-100%)	1	1	1
One-Off Errors	1	6 (+500%)	1	0 (-100%)	4 (+300%)	0	1 (+100%)	1 (+100%)	0	3 (+300%)	0	2 (+200%)	1 (+100%)	1 (+100%)	1 (+100%)
Total	82	122 (+49%)	106 (+29%)	109 (+33%)	118 (+44%)	81	96 (+19%)	94 (+16%)	95 (+17%)	124 (+53%)	89	99 (+11%)	102 (+15%)	98 (+10%)	111 (+25%)

PyPDF2.errors.DeprecationError is shortened to *DeprecationError**. *sqlite3.OperationalError* is shortened to *OperationalError**.

the resulting errors. Error types were categorized following prior work [27, 34]. SOEN-101 [27] classified failures using Python’s official exception taxonomy⁴, while ChatDev [34] identified compilation outcomes and logged compiler-reported errors.

Minor syntax errors (e.g., missing or extra parentheses or punctuation) were manually corrected and tests rerun. Repeated instances of the same error type within a class were counted once to avoid overcounting. We additionally introduced *Time Limit Exceeded* (execution > 8 minutes) and *One-Off Errors*, which group rare failures (e.g., *ModuleNotFoundError*, *re.error*, *RuntimeError*).

Results were summarized as an error frequency chart (Table 3), where each value denotes the number of classes exhibiting that error.

Results: *ValueError*, *AssertionError*, and *TypeError* account for most failures and increase sharply under Waterfall workflows ($\approx +40\%$ to $>+700\%$). *ValueError* shows the largest growth, rising from roughly 7 to about 25 cases, while *AssertionError* often more than doubles and *TypeError* typically doubles. In contrast, simpler runtime errors decline substantially: *ZeroDivisionError* nearly disappears, *IndexError* often drops to zero, and *NameError* decreases by over 90%. Less frequent errors, including *KeyError*, *OperationalError*, and grouped *One-Off Errors*, increase modestly.

Overall runtime errors increase by 10–53% under Waterfall workflows compared to *RawPrompt*. The increase is largest for *GPT-4o-Mini* (+49%), moderate for *DeepSeek-Chat* (+16–19%), and smallest for *Claude-3.5-Haiku* (+10–25%). Removing the testing stage (w/o-Test) consistently yields the highest failure rates across all models (up to +53%), highlighting the importance of test awareness.

The full Waterfall workflow produces the most runtime errors, while stage removal has mixed, model-dependent effects. In most cases, the full Waterfall setup increases failures relative to *RawPrompt* (e.g., +49% for *GPT-4o-Mini*, +19% for *DeepSeek-Chat*). Omitting requirements or design leads to minor and inconsistent changes, whereas skipping testing consistently causes the largest error increase across models. Overall, early-stage omissions have limited impact, but removing testing substantially degrades reliability.

RQ2 Summary: Waterfall workflows increase runtime errors by 10–53% compared to *RawPrompt*. *ValueError*, *AssertionError*, and *TypeError* dominate the failures, while *ZeroDivisionError* and other low-level mistakes decline. DeepSeek-Chat is the comparatively more stable model, whereas GPT-4o-Mini and Claude-3.5-Haiku show greater sensitivity to workflow design, indicating that structured prompting must be carefully tuned for each model.

4.3 RQ3: What are the underlying causes behind these failures?

Motivation: While RQ2 identified the types of runtime errors in multi-agent code generation, it did not explain their causes. This research question investigates the underlying reasons by analyzing interactions across workflow stages, including information loss or inconsistency between agents, incorrect implementation of design decisions, and limitations in model reasoning. This analysis can clarify how workflow structure, model behavior, and stage coordination affect overall reliability.

Approach: We isolated all failing classes from the 1,500 RQ2 samples (100 ClassEval tasks $\times$ 3 models $\times$ 5 workflows) and manually inspected each to determine the root cause. Generated code was compared against three references: *ClassEval Task Description*. We assessed whether the code satisfied the stated requirements and examined task descriptions for ambiguities, inconsistencies, or missing details. *Unit Tests*. We analyzed failing tests to identify missing functionality, logic errors, or incorrect case handling, and verified consistency between the tests and the task descriptions. *Reference Implementation*. We compared generated code against the canonical reference implementation to identify logical, data-flow, and structural deviations causing incorrect behavior, and assessed whether the reference contained details beyond the problem description.

Following prior LLM fault-analysis work [34, 42], we annotated each issue using a structured error taxonomy grouped into broader types (Table 4). Each failure was assigned a primary cause, with cross-references when multiple factors contributed. This analysis was applied to GPT-4o-Mini, DeepSeek-Chat, and Claude-3.5

⁴<https://docs.python.org/3/library/exceptions.html>

Table 4: Taxonomy of error categories with corresponding motivations describing the underlying causes of failures in class-level code generation.

Error Category	Motivation
Missing Code	Code components such as variables, functions, or classes are missing, renamed, or only partially implemented.
Renamed Variable	A variable name has been changed from what the specification requires.
Missing Variable	A required variable from the specification is missing in the code.
Missing Function	A function is called but not defined anywhere in the code.
Renamed Function	A function was renamed, but the test still uses the old name.
Missing Class	A class used in the code/test is not defined.
Renamed Class	A class was renamed but tests still use the old name.
Missing Import	Code uses a module or function without importing it.
Incomplete Implementation	Only part of the instructed functionality is implemented.
Return Mismatch	Returned outputs differ from expected results due to incorrect arity, order, type, or format.
Arity Mismatch	Function returns too many or too few values (e.g., 3 items instead of 2).
Order Mismatch	Function returns values in the wrong order (e.g., (b, a) instead of (a, b)).
Type Mismatch	The return value is of the wrong Python type (e.g., list instead of tuple).
Format Mismatch	The return value has incorrect formatting (e.g., casing, spacing, symbols).
Input Validation	The code fails to correctly handle user input due to being too strict, too lenient, or logically flawed.
Overrestrictive Validation	Adds unnecessary constraints not required by the problem.
Faulty Validation	Validation logic is incorrect or incomplete.
Missing Input Validation	No validation occurs where it should.
Semantic Failure	Logical or algorithmic errors causing functionality to deviate from intended behavior.
Spec Violation	Code violates explicit problem instructions.
Signature Mismatch	Function parameters differ from the expected specification.
Wrong Algorithm	Code implements an incorrect algorithm or formula.
Wrong Edge Case Handling	Fails to handle boundary cases properly.
Timeout	Code runs indefinitely or exceeds time limits.
Syntax Error	Code cannot compile or parse due to syntax issues.
Integration Error	A newly added feature conflicts with existing code.
Dataset	Errors arise from ambiguous, flawed, or inconsistent problem specifications or test data.
Faulty Spec	The problem description is incorrect or misleading.
Faulty Test	Test cases contain incorrect or inconsistent logic.
Missing Import (Test)	Test scripts fail due to missing imports.
Spec-Test Mismatch	Tests contradict the written problem specification.
Environment	Failures caused by dependency, compatibility, or versioning issues.
Version Incompatibility	Uses unsupported features in the given Python version.
Undeclared Dependency	Uses undeclared external packages.
Deprecated Dependency	Uses outdated or removed dependencies.

-Haiku under agent-based Waterfall workflows. The taxonomy spans common LLM code errors [45], including structural issues (*Missing Code*, *Return Mismatch*), reasoning flaws (*Input Validation*, *Semantic Failure*), and data or setup problems (*Dataset*, *Environment*).

Results: Overall, Dataset-related issues were the most prevalent failure source (47 – 62 cases), followed by Semantic Failure (50 – 55) and Return Mismatch (21 – 34). As shown in Table 5, these categories dominated across all Waterfall variants. This pattern highlights persistent challenges in algorithmic reasoning and handling ambiguous inputs. Notably, *Input Validation* errors, rare in the RawPrompt baseline (2 cases), rose to 18 cases under Waterfall workflows. This suggests that while structured workflows improve validation coverage, tighter coordination and stricter specification enforcement can surface deeper logical constraints.

Failure rates related to reasoning and output were significant across all models. *Semantic Failure* increased sharply: Claude-3.5-Haiku rose from 18 to 54 cases ($\approx 200\%$), DeepSeek-Chat from 29 to 53 ($\approx 83\%$), and GPT-4o-Mini from 35 to 55 ($\approx 57\%$), while *Return Mismatch* remained frequent (21–34 cases), mainly

due to *Type* and *Format* issues ($\approx 75\text{--}83\%$ of return errors). Most stemmed from *Wrong Algorithm* (19 – 30 cases) and *Wrong Edge Case Handling* (4 – 12), indicating weaknesses in logical precision and boundary reasoning. Notably, GPT-4o-Mini *Specification Violations* rose (2 $\rightarrow$ 12). *Return Mismatch* increased for GPT-4o-Mini (13 – 22%) and DeepSeek-Chat (21 – 34 cases), while Claude-3.5-Haiku surged 108%. Despite fewer *Arity Mismatches*, stage coordination introduced new formatting inconsistencies, revealing a trade-off between structural rigor and semantic reliability.

Input Validation errors surged under Waterfall configurations. GPT-4o-Mini reached maximum *Input Validation* (16 cases) and *Dataset* failures (62 cases) when omitting the *requirements* and *design* phases, respectively. DeepSeek-Chat validation errors peaked (18) when the *testing* phase was excluded. In contrast, Claude-3.5-Haiku reached its highest error counts (11 validation, 59 dataset) in the *full Waterfall* setup. These were largely categorized as *Overrestrictive Validation* (unnecessary checks) or *Faulty Validation* (incorrect conditions), while *Missing Validation* errors appeared rarely.

Dataset errors varied by model: GPT-4o-Mini showed a moderate increase ($\approx 32\%$, reaching 62 cases), while DeepSeek-Chat remained stable (49 – 52 cases). Conversely, Claude-3.5-Haiku experienced a sharp rise from 24 to 59 cases ($\approx 146\%$), driven primarily by *Faulty Specifications* and *Faulty Tests*.

Missing Code errors dropped drastically under Waterfall, falling from 4 – 50 cases in RawPrompt to as few as 1 – 2 cases. Claude-3.5-Haiku showed the most significant improvement, with a reduction of $> 95\%$. This indicates that multi-stage structures effectively prevent incomplete implementations.

Environment issues remained consistently low (1 – 5 cases). Claude-3.5-Haiku eliminated these entirely in full Waterfall setups, while GPT-4o-Mini and DeepSeek-Chat showed only minor fluctuations. Issues such as *Version Incompatibility* and *Undeclared Dependency* were rare, suggesting models generally manage dependencies effectively and environments correctly within structured frameworks.

RQ3 Summary: Waterfall workflows reduce structural errors such as *Missing Code* but increase reasoning and validation issues such as *Semantic Failure* and *Return Mismatch*. *Dataset* related errors remain the most common overall. Each model shows trade-offs; GPT-4o-Mini and DeepSeek-Chat improve completeness, while Claude-3.5-Haiku faces higher logic and validation failures.

5 Discussion

5.1 Implications

For Developers. Waterfall-style, role-based agents can improve code structure and maintainability by encouraging clearer design and validation. But this extra structure often adds checks or constraints not required by the specification, which can lower execution success and *Pass@1*. In benchmarks, these failures happen because strict spec matching matters more than defensive or production-style logic. *Practical guidance.* (i) *Spec-driven validation:* check only what the task explicitly requires. (ii) *Stage prioritization:* keep requirement clarification and test planning, but make design and

Table 5: Qualitative analysis of error categories across models and Waterfall variants. Each cell shows the relative frequency of errors compared to the *RawPrompt* baseline, where red indicates an increase and green indicates a decrease.

Error Category	GPT-4o-Mini					DeepSeek-Chat					Claude-3.5-Haiku				
	Raw	Waterfall				Raw	Waterfall				Raw	Waterfall			
		Full	w/o-Req.	w/o-Des.	w/o-Test.		Full	w/o-Req.	w/o-Des.	w/o-Test.		Full	w/o-Req.	w/o-Des.	w/o-Test.
Missing Code	4	1	4	0	10	1	0	0	1	1	50	2	1	1	17
Renamed Variable	1	1	2	0	0	0	0	0	0	0	0	0	0	0	0
Missing Variable	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0
Missing Function	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0
Renamed Function	0	0	0	0	6	1	0	0	0	1	0	0	0	0	0
Missing Class	1	0	0	0	0	0	0	0	0	0	49	2	1	1	17
Renamed Class	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Missing Import	2	0	0	0	0	0	0	0	0	0	1	0	0	0	0
Incomplete Implementation	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0
Return Mismatch	23	26	28	27	26	21	23	25	34	32	13	22	27	27	26
Arity Mismatch	4	1	6	3	3	5	4	5	7	3	3	5	5	4	4
Order Mismatch	1	2	1	0	1	1	0	1	1	1	1	0	3	3	1
Type Mismatch	8	13	14	13	16	8	7	9	11	13	3	5	7	9	7
Format Mismatch	10	10	7	11	6	7	12	10	15	14	6	12	12	11	14
Input Validation	2	14	16	10	13	3	9	8	8	18	1	10	6	3	11
Overrestrictive Validation	0	9	10	6	7	0	6	5	4	12	0	6	3	2	7
Faulty Validation	2	5	6	4	6	1	3	2	2	6	1	3	3	1	4
Missing Input Validation	0	0	0	0	0	2	0	1	2	0	0	1	0	0	0
Semantic Failure	35	51	49	55	50	29	52	50	47	53	18	43	54	51	39
Spec Violation	2	12	7	9	7	1	5	3	3	9	0	4	4	3	2
Signature Mismatch	0	0	1	2	3	0	0	0	0	1	0	0	0	1	0
Wrong Algorithm	22	25	25	25	23	19	24	22	24	23	13	29	26	30	23
Wrong Edge Case Handling	9	10	8	12	8	4	5	5	5	1	4	6	9	7	7
Timeout	2	1	1	0	0	2	0	1	0	2	1	0	1	1	1
Syntax Error	0	0	3	6	6	3	14	17	13	14	0	1	12	6	3
Integration Error	0	3	4	1	3	0	4	2	1	3	0	3	2	3	3
Dataset	47	59	51	62	52	52	49	52	48	52	24	49	59	51	44
Faulty Spec	15	22	20	26	21	19	19	21	21	21	6	22	24	22	20
Faulty Test	6	7	7	7	6	9	8	9	7	9	5	8	10	7	5
Missing Import (Test)	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0
Environment	1	1	2	1	5	1	1	2	1	2	1	0	1	1	3
Version Incompatibility	0	0	2	1	3	0	0	0	0	1	0	0	0	0	0
Undeclared Dependency	0	1	0	0	1	0	0	1	0	0	1	0	0	0	3
Deprecated Dependency	1	0	0	0	1	1	1	1	0	1	0	0	1	1	0

validation less rigid when correctness matters most. (iii) *Contract checks*: confirm interfaces, return types, and edge cases match the task definition.

For Researchers and Benchmark Designers. Structured workflows can change the types of failures, shifting them from simple mistakes to reasoning or contract errors. *Implications. For researchers*: build adaptive pipelines, add cross-stage consistency checks, and prioritize benchmark specifications when stages conflict. *For benchmark designers*: define clear contracts, report both correctness and code quality, and mark when extra validation is acceptable.

5.2 Threats to Validity

Internal Validity. Our qualitative analysis for RQ3 is partly subjective and may not always identify the exact cause of errors. Future work could use LLM-based evaluators to help diagnose errors and reduce manual effort [24]. Error categorization may also vary slightly across models, so we used random sampling, consistent labels, and reviewed samples within the same period to reduce bias.

External Validity. We mainly use Pass@1 to measure functional correctness. Because it evaluates only the first generated solution, it can be affected by randomness and may not fully capture model ability. Still, Pass@1 is widely used in code generation research, which supports comparison with prior work such as Codex [11], AlphaCode [26], and CodeT5+ [39].

Construct Validity. We use LLM agents to generate unit tests

for the Testing activity, following prior multi-agent software engineering work [19, 27, 34]. These tests may not cover all program behaviours. Also, function-level Pass@1 may not reflect correctness well for class-level tasks, where methods depend on each other. As a result, correct methods may be penalized, producing lower and less representative scores than class-level evaluation [27].

6 Conclusion

In this paper, we have empirically evaluated multi-agent LLM workflows within the Waterfall model for class-level code generation using ClassEval. The controlled experiments show that Waterfall workflows do not consistently improve functional correctness but yield cleaner, more maintainable code with fewer structural issues such as missing methods. Runtime and logical errors increase slightly, often due to stricter validation, semantic failures, and incorrect returns. Among development activities, Testing has the biggest impact, improving verification but reducing maintainability, while Requirement and Design play minor roles. Overall, the complete Waterfall workflow produces structured and readable code but limits flexibility and correctness, revealing a trade-off between disciplined process control and adaptive reasoning in multi-agent LLM systems.

References

- [1] Windsurf AI. 2025. Windsurf. <https://windsurf.ai>.
- [2] Maha Alharbi and Mohammad Alshayeb. 2025. Automatic Code Generation Techniques: A Systematic Literature Review. *Automated Software Engineering* 33, 1 (Sept. 2025), 4. doi:10.1007/s10515-025-00551-3

- [3] Anonymous. 2025. Repository & dataset. <https://anonymous.4open.science/r/LLMCLWA-3XB/README.md>
- [4] Anthropic. 2025. Claude. <https://claude.ai>.
- [5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [6] Berkay Berabi, Alexey Gronskey, Veselin Raychev, Gishor Sivanrupan, Victor Chibotaru, and Martin Vechev. 2024. Deepcode AI fix: Fixing security vulnerabilities with large language models. *arXiv preprint arXiv:2402.13291* (2024).
- [7] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv:2403.17134*
- [8] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2025. Why Do Multi-Agent LLM Systems Fail? *arXiv preprint arXiv:2503.13657* (2025).
- [9] Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code search is all you need? improving code suggestions with code search. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 1–13.
- [10] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2024. Reasoning runtime behavior of a program with llm: How far are we? *arXiv preprint arXiv:2403.16437* (2024).
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [12] Cursor. 2025. Cursor. <https://www.cursor.sh>.
- [13] DeepSeek. 2025. DeepSeek Chat. <https://chat.deepseek.com>.
- [14] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A survey on code generation with llm-based agents. *arXiv preprint arXiv:2508.00083* (2025).
- [15] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).
- [16] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024).
- [17] GitHub. 2025. GitHub Copilot. <https://github.com/features/copilot>.
- [18] Xinyi He, Jiaru Zou, Yun Lin, Mengyu Zhou, Shi Han, Zejian Yuan, and Dongmei Zhang. 2024. Cocost: Automatic complex code generation with online searching and correctness testing. *arXiv preprint arXiv:2403.13583* (2024).
- [19] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations, ICLR.
- [20] Shuyang Hou, Haoyue Jiao, Zhangxiao Shen, Jianyuan Liang, Anqi Zhao, Xiaopu Zhang, Jianxun Wang, and Huayi Wu. 2024. Chain-of-Programming (CoP): Empowering Large Language Models for Geospatial Code Generation. *arXiv preprint arXiv:2411.10753* (2024).
- [21] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).
- [22] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1424–1446.
- [23] Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. 2019. Spoc: Search-based pseudocode to code. *Advances in Neural Information Processing Systems* 32 (2019).
- [24] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llm-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [25] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.
- [26] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Aultume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158 [arXiv:https://www.science.org/doi/pdf/10.1126/science.abq1158](https://www.science.org/doi/pdf/10.1126/science.abq1158)
- [27] Feng Lin, Dong Jae Kim, et al. 2024. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. *arXiv preprint arXiv:2403.15852* (2024).
- [28] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv:2409.00899*
- [29] Xinbei Ma, Yeyun Gong, Pengcheng He, Nan Duan, et al. 2023. Query rewriting in retrieval-augmented large language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- [30] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *arXiv:2303.17651*
- [31] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. Clarifygpt: Empowering llm-based code generation with intention clarification. *arXiv preprint arXiv:2310.10996* (2023).
- [32] Minh Huynh Nguyen, Thang Phan Chau, Phong X Nguyen, and Nghi DQ Bui. 2025. Agilecoder: Dynamic collaborative agents for software development based on agile methodology. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 156–167.
- [33] OpenAI. 2025. ChatGPT. <https://chat.openai.com>.
- [34] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2023. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
- [35] Md Nakhla Rafi, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. The impact of input order bias on large language models for software fault localization. *arXiv preprint arXiv:2412.18750* (2024).
- [36] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, et al. 2023. Pangu-coder2: Boosting large language models for code with ranking feedback. *arXiv preprint arXiv:2307.14936* (2023).
- [37] Tabnine. 2025. Tabnine. <https://www.tabnine.com>.
- [38] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
- [39] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).
- [40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [41] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [42] Pengyu Xue, Linhao Wu, Zhen Yang, Chengyi Wang, Xiang Li, Yuxiang Zhang, Jia Li, Ruikai Jin, Yifei Pei, Zhaoyan Shen, et al. 2025. ClassEval-T: Evaluating Large Language Models in Class-Level Code Translation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1421–1444.
- [43] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793*
- [44] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. ThinkRepair: Self-Directed Automated Program Repair. In *ISSTA 2024: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, United States, 1274–1286.
- [45] Adam Yuen, John Pangas, Md Mainul Hasan Polash, and Ahmad Abdellatif. [n. d.]. Prompting Matters: Assessing the Effect of Prompting Techniques on LLM-Generated Class Code. ([n. d.]).
- [46] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1592–1604. doi:10.1145/3650212.3680384
- [47] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2023. Unifying the perspectives of nlp and software engineering: A survey on language models for code. *arXiv preprint arXiv:2311.07989* (2023).
- [48] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372* (2023).
- [49] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).